CS 112

Introduction to Data Structures

WEEK 14

Hash Tables and Sorting

Eric Araújo

Calvin University · Fall 2026

This Week

1. Computing where an item lives
2. Hash functions
3. Collisions **KEY**
4. Load factor and the time–space trade
5. Selection sort
6. Insertion sort
7. Where the course has taken you

What If We Just Computed the Index?

A tree finds an item in O(lg n) by comparing. An array finds index `i` in O(1) by arithmetic.

> So: turn the *key itself* into an index. No comparisons, no walking — one calculation and you're there. That's a **hash table**.

A Hash Function

```
unsigned hash(const string &key, unsigned capacity) {
    unsigned sum = 0;
    for (char c : key) {
        sum = sum * 31 + c;        // mix the characters
    }
    return sum % capacity;         // fold into the table
}
```

- **Deterministic** — same key, same slot, every time
- **Fast** — otherwise you've lost the advantage
- **Spreads out** — keys should scatter across the table

> The `% capacity` is what forces an unbounded key space into a fixed number of slots — and that is exactly why collisions are unavoidable.

Collisions

slot 0		—
slot 1	"ada" → "bob"	both hashed to 1
slot 2		"grace"

Two keys, one slot. With 4 billion possible strings and 100 slots, this is not bad luck — it's arithmetic.

Chaining — each slot holds a linked list of everything that landed there

Open addressing — on a clash, probe forward for the next free slot

TALK TO YOUR NEIGHBOR · TTYN

With chaining, what is the worst case for `find`?

A. O(1) — hashing is always constant

B. O(lg n) — the chain is a tree

C. O(n) — every key could hash to the same slot

D. $O(n^2)$

Load Factor

```
load factor = items / slots
```

- Low (say 0.5) — short chains, fast lookups, memory sitting empty
- High (say 5.0) — memory well used, chains long, lookups slower
- Most implementations **rehash** — double the table and redistribute — around 0.75

> This is the **time–space trade-off** in its purest form: buy speed with empty slots. Rehashing is O(n), amortised across the inserts — exactly the doubling argument from week 5.

Where Hash Tables Sit

Operation	Hash table	Balanced tree	Sorted array
find	**O(1)** avg	O(lg n)	O(lg n)
insert	**O(1)** avg	O(lg n)	O(n)
remove	**O(1)** avg	O(lg n)	O(n)
sorted listing	O(n lg n)	**O(n)**	**O(n)**
worst case	O(n)	O(lg n)	O(lg n)

Fastest on average, and the only one with no sense of order. Every row of this table is a design decision you can now make on purpose.

Selection Sort

```
for (int i = 0; i < n - 1; ++i) {
    int smallest = i;
    for (int j = i + 1; j < n; ++j) {
        if (a[j] < a[smallest]) { smallest = j; }
    }
    swap(a[i], a[smallest]);
}
```

Find the smallest of what's left, swap it into place, repeat.

Nested loops over n → **O(n²)**, always. It does the same work on sorted input as on random input, but it makes at most n−1 swaps.

Insertion Sort

```
for (int i = 1; i < n; ++i) {
    Item key = a[i];
    int j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];      // shift right
        --j;
    }
    a[j + 1] = key;           // drop it in
}
```

Take the next item, slide it back until it's in place — the way you sort a hand of cards.

> Worst case $O(n^2)$, but on **already-sorted** input the inner loop never runs: $O(n)$.

TALK TO YOUR NEIGHBOR · TTYN

Your data is almost sorted already. Which of the two is better?

A. Selection sort — it always does the same work

B. Insertion sort — it approaches O(n) when little is out of place

C. Identical — both are $O(n^2)$

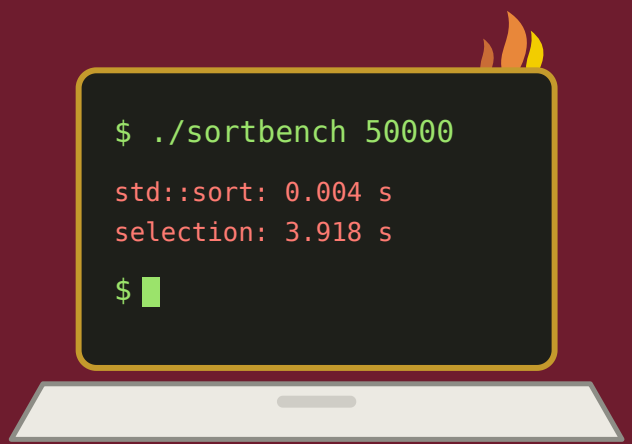
D. Neither works on partly sorted data

Comparing the Two

	Selection sort	Insertion sort
Best case	$O(n^2)$	**O(n)**
Worst case	$O(n^2)$	$O(n^2)$
Swaps / writes	**O(n)**	$O(n^2)$
Stable?	no	**yes**

Neither is what you'd ship — `std::sort` is O(n lg n). But every fast sort is built from ideas you can now read, and both of these are the right choice for a small enough n.

Demo



Racing selection sort against std::sort.

(n = 50,000, and then we wait)

Fourteen Weeks

Structure	Good at	Bad at
Array / vector	indexing, appending	inserting at the front
Linked list	inserting and removing anywhere you already are	indexing
Stack / queue	one disciplined end (or two)	looking in the middle
BST	ordered search — when balanced	sorted input, unless balanced
AVL tree	guaranteed O(lg n)	more code, more memory
Hash table	raw lookup speed	any question about order

You built every one of these. The skill you're leaving with isn't any single structure — it's asking *which operations will this code do a million times?* before choosing.

Week 14 Recap

- A hash function turns a key into an index — O(1) on average
- Collisions are inevitable; **chaining** and **open addressing** handle them
- Load factor trades memory for speed; rehashing is the doubling trick again
- Selection sort is $O(n^2)$ always; insertion sort is O(n) on nearly-sorted data
- Choose the structure by the operations you repeat most

Next: a23 · final exam →

This deck stands on earlier CS112 materials by
Joel Adams and **Victor Norman** · adapted and extended by **Eric Araújo**